36.1 macOS 应用（OpenClaw.app）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~175k tokens，输出 ~8k tokens（本节）

OpenClaw 的 macOS 应用是一个 **菜单栏应用**（Menu Bar App），常驻系统菜单栏，提供语音唤醒、对话模式、Gateway 进程管理等功能。它用 Swift 6.2 + SwiftUI 编写，通过 WebSocket 连接到 Gateway 实例。

36.1.1 Swift + SwiftUI 架构

项目结构

macOS 应用位于 `apps/macos/`，使用 Swift Package Manager（SPM）管理依赖：

```swift
// apps/macos/Package.swift
let package = Package(
    name: "OpenClaw",
    platforms: [.macOS(.v15)],   // 最低 macOS 15 (Sequoia)
    products: [
        .library(name: "OpenClawIPC", targets: ["OpenClawIPC"]),
        .library(name: "OpenClawDiscovery", targets: ["OpenClawDiscovery"]),
        .executable(name: "OpenClaw", targets: ["OpenClaw"]),        // 主应用
        .executable(name: "openclaw-mac", targets: ["OpenClawMacCLI"]), // CLI 工具
    ],
    dependencies: [
        .package(url: "MenuBarExtraAccess", exact: "1.2.2"),  // 菜单栏扩展
        .package(url: "swift-subprocess", from: "0.1.0"),     // 进程管理
        .package(url: "swift-log", from: "1.8.0"),
        .package(url: "Sparkle", from: "2.8.1"),              // 自动更新
        .package(url: "Peekaboo", branch: "main"),            // 自动化
        .package(path: "../shared/OpenClawKit"),               // 共享库
    ],
)
```

项目由四个 target 组成：

Target	类型	职责
`OpenClaw`	可执行文件	菜单栏主应用
`OpenClawMacCLI`	可执行文件	`openclaw-mac` 命令行工具
`OpenClawIPC`	库	进程间通信
`OpenClawDiscovery`	库	Gateway 服务发现

源码规模

`Sources/OpenClaw/` 目录包含约 **191 个 Swift 文件**，覆盖应用的所有功能模块：

```
Sources/OpenClaw/
├── AppState.swift                    # 全局状态（@Observable）
├── MenuBar.swift                     # @main 入口 + MenuBarExtra
├── GatewayConnection.swift           # WebSocket 客户端（actor）
├── GatewayProcessManager.swift       # Gateway 进程生命周期
├── VoiceWake*.swift（15+ 文件）      # 语音唤醒系统
├── TalkMode*.swift（6 文件）         # 对话模式
├── Canvas*.swift（8 文件）           # Canvas 表面
├── Channels*.swift（8 文件）         # 通道管理 UI
├── Onboarding*.swift（8 文件）       # 首次引导
├── *Settings.swift（12 文件）        # 各功能的设置面板
├── *Store.swift（8 文件）            # 数据存储
└── ...
```

状态管理

应用采用 Swift 5.9+ 的 `@Observable` 宏（Observation 框架）管理全局状态：

```swift
// apps/macos/Sources/OpenClaw/AppState.swift
@MainActor
@Observable
final class AppState {
    enum ConnectionMode: String {
        case unconfigured
        case local     // 本机运行 Gateway
        case remote    // 连接远程 Gateway
    }

    var isPaused: Bool { didSet { UserDefaults.standard.set(isPaused, forKey: key) } }
    var launchAtLogin: Bool { didSet { AppStateStore.updateLaunchAtLogin(enabled: self)
    var swabbleEnabled: Bool { didSet { VoiceWakeRuntime.shared.refresh(state: self) }
    var swabbleTriggerWords: [String] { didSet { ... } }
    // ... 30+ 可观察属性
}
```

> **衍生解释 — @Observable 与 @MainActor**
>
> `@Observable` 是 Swift 5.9 引入的宏，替代旧的 `ObservableObject` 协议。它在编译时自动为属性生成变更跟踪代码，SwiftUI 视图只在真正使用到的属性变化时才重绘——比 `@Published` 的"任何属性变化都通知"更精准。`@MainActor` 确保该类的所有方法都在主线程执行，这对 UI 状态管理来说是安全必要的。

36.1.2 菜单栏控制（Menu Bar）

入口点

应用的入口是一个 `MenuBarExtra` ——macOS 菜单栏中的一个常驻图标：

```swift
// apps/macos/Sources/OpenClaw/MenuBar.swift
@main
struct OpenClawApp: App {
    @State private var state: AppState
    private let gatewayManager = GatewayProcessManager.shared
    private let controlChannel = ControlChannel.shared

    var body: some Scene {
        MenuBarExtra {
            MenuContent(state: self.state, updater: ...)
        } label: {
            CritterStatusLabel(
                isPaused: self.state.isPaused,
                isWorking: self.state.isWorking,
                gatewayStatus: self.gatewayManager.status,
                animationsEnabled: self.state.iconAnimationsEnabled,
                ...)
        }
        .menuBarExtraStyle(.menu)
        .menuBarExtraAccess(isPresented: self.$isMenuPresented) { item in
            MenuSessionsInjector.shared.install(into: item)
        }

        Settings {
            SettingsRootView(state: self.state, ...)
        }
    }
}
```

菜单栏图标是一个动画化的"小生物"（Critter），它的状态会反映 Gateway 的运行情况：

图标状态	含义
常态动画	Gateway 正常运行
暂停/灰色	用户手动暂停
睡眠	Gateway 休眠中
工作中	Agent 正在处理任务
耳朵竖起（earBoost）	语音唤醒激活

连接模式

应用支持两种 Gateway 连接模式：

- **本地模式（local）**：应用自己启动和管理一个 Gateway 子进程
- **远程模式（remote）**：连接到网络上的另一个 Gateway 实例

模式切换由 `ConnectionModeCoordinator` 协调：

```
.onChange(of: self.state.connectionMode) { _, mode in
    Task { await ConnectionModeCoordinator.shared.apply(
        mode: mode, paused: self.state.isPaused) }
}
```

36.1.3 Voice Wake + Push-to-Talk

语音唤醒系统

语音唤醒是 macOS 应用的核心特性之一。它使用 Apple 的 **Speech 框架**（`SFSpeechRecognizer`）进行实时语音识别，监听用户定义的触发词。

```
// apps/macos/Sources/OpenClaw/VoiceWakeRuntime.swift
actor VoiceWakeRuntime {
    static let shared = VoiceWakeRuntime()

    enum ListeningState { case idle, voiceWake, pushToTalk }

    private var recognizer: SFSpeechRecognizer?
    private var audioEngine: AVAudioEngine?
    private var recognitionRequest: SFSpeechAudioBufferRecognitionRequest?
    // ...

    // 可调参数
    private let silenceWindow: TimeInterval = 2.0        // 语音后静默阈值
    private let triggerOnlySilenceWindow: TimeInterval = 5.0
    private let captureHardStop: TimeInterval = 120.0     // 最大捕获时长
    private let minSpeechRMS: Double = 1e-3               // 最小语音 RMS
    private let speechBoostFactor: Double = 6.0           // 噪声底以上倍数
}
```

> **衍生解释 — SFSpeechRecognizer 与 RMS**
>
> `SFSpeechRecognizer` 是 Apple 的语音识别 API，支持实时流式识别。它将麦克风捕获的音频缓冲区（`AVAudioPCMBuffer`）转换为文本。RMS（Root Mean Square，均方根）是衡量音频信号强度的常用指标——OpenClaw 通过比较当前 RMS 与噪声底（`noiseFloorRMS`）来判断用户是否在说话。

语音唤醒的流程：

1. 启动 `AVAudioEngine` 持续捕获音频
2. 将音频缓冲区送入 `SFSpeechAudioBufferRecognitionRequest` 进行实时识别
3. 识别回调中检查文本是否包含触发词（如 "Hey OpenClaw"）
4. 触发后进入"捕获模式"——继续录音直到检测到 2 秒静默
5. 将捕获的语音发送给 Gateway

音频引擎被 **延迟创建**：只有在语音唤醒功能真正开启时才初始化 `AVAudioEngine`，因为仅仅创建它就会导致蓝牙耳机切换到低质量的 headset profile。

36.1.4 Talk Mode 覆盖层

Talk Mode 是一种全屏覆盖的对话模式，类似于语音助手的"对话界面"：

```swift
// apps/macos/Sources/OpenClaw/TalkModeController.swift
@MainActor
@Observable
final class TalkModeController {
    static let shared = TalkModeController()

    private(set) var phase: TalkModePhase = .idle
    private(set) var isPaused: Bool = false

    func setEnabled(_ enabled: Bool) async {
        if enabled {
            TalkOverlayController.shared.present()   // 显示全屏覆盖层
        } else {
            TalkOverlayController.shared.dismiss()
        }
        await TalkModeRuntime.shared.setEnabled(enabled)
    }

    func updatePhase(_ phase: TalkModePhase) {
        self.phase = phase
        TalkOverlayController.shared.updatePhase(phase)
        // 同步状态到 Gateway
        Task { await GatewayConnection.shared.talkMode(
            enabled: AppStateStore.shared.talkEnabled,
            phase: effectivePhase) }
    }
}
```

Talk Mode 的阶段（phase）包括：idle（空闲）、listening（监听中）、thinking（思考中）、speaking（播报中）等。每个阶段变化都同步到 Gateway，Gateway 可以据此协调其他客户端的行为。

36.1.5 远程 Gateway 控制

在远程模式下，macOS 应用通过 SSH 隧道或直接 WebSocket 连接到远程 Gateway：

```swift
// apps/macos/Sources/OpenClaw/GatewayConnection.swift
actor GatewayConnection {
    static let shared = GatewayConnection()

    enum Method: String, Sendable {
        case agent, status, health
        case channelsStatus = "channels.status"
        case configGet = "config.get"
        case configSet = "config.set"
        case chatHistory = "chat.history"
        case chatSend = "chat.send"
        case chatAbort = "chat.abort"
        case talkMode = "talk.mode"
        case voicewakeGet = "voicewake.get"
        case voicewakeSet = "voicewake.set"
        // ... 20+ RPC 方法
    }
}
```

`GatewayConnection` 使用 Swift 的 `actor` 并发模型确保线程安全——所有 WebSocket 操作自动串行化，无需手动加锁。远程连接支持通过 `RemoteTunnelManager` 建立 SSH 隧道。

36.1.6 macOS 权限与 TCC

macOS 有严格的 **TCC（Transparency, Consent, and Control）** 权限系统。OpenClaw 需要多项系统权限才能完整工作：

```swift
// apps/macos/Sources/OpenClaw/PermissionManager.swift
enum PermissionManager {
    static func ensure(_ caps: [Capability], interactive: Bool) async -> [Capability: B
        var results: [Capability: Bool] = [:]
        for cap in caps {
            results[cap] = await ensureCapability(cap, interactive: interactive)
        }
        return results
    }
}
```

所需权限包括：

权限	用途	API
麦克风	语音唤醒/对话模式	`AVCaptureDevice.requestAccess(for: .audio)`
语音识别	语音转文本	`SFSpeechRecognizer.requestAuthorization()`
摄像头	视觉能力	`AVCaptureDevice.requestAccess(for: .video)`
屏幕录制	截屏/录屏工具	`CGPreflightScreenCaptureAccess()`
AppleScript	自动化操作	AppleScript 权限
辅助功能	键盘监听	Accessibility API
通知	消息提醒	`UNUserNotificationCenter.requestAuthorization()`
位置	位置查询工具	`CLLocationManager.requestAlwaysAuthorization()`

> **衍生解释 — TCC (Transparency, Consent, and Control)**
>
> TCC 是 macOS 和 iOS 的隐私保护框架。应用访问敏感资源（摄像头、麦克风、文件夹等）前必须获得用户明确授权。权限状态存储在系统数据库 `TCC.db` 中，用户可以在系统偏好设置的"隐私与安全性"面板中管理。对开发者来说，必须在 Info.plist 中声明使用说明（`NSMicrophoneUsageDescription` 等），否则权限请求会静默失败。

`PermissionManager` 在交互模式下会引导用户授权（弹出系统对话框或跳转到设置页面），非交互模式下只检查不请求。

> **v2026.3.9 更新** —— macOS 应用在 Chat 界面新增了两项功能：
>
> 1. **Chat 模型选择器（Model Picker）**：在 Chat 界面顶部新增模型切换入口，用户可以在对话过程中直接切换不同的 AI 模型，无需进入设置页面。
> 2. **思考过程持久化（Thinking Persistence）**：当使用支持推理模式的模型（如 Claude 的 extended thinking）时，模型的思考过程现在会持久化保存到本地，重启应用后仍可查看历史对话中的推理链条。

本节小结

1. **macOS 应用是菜单栏常驻应用**，使用 Swift 6.2 + SwiftUI + SPM 构建，支持本地和远程两种 Gateway 连接模式。
2. **菜单栏图标**是动画化的状态指示器，反映 Gateway 运行状态、工作状态和语音唤醒状态。

3. **语音唤醒**基于 Apple Speech 框架实现实时语音识别，支持自定义触发词，使用 RMS 能量检测进行语音活动判定。
4. **Talk Mode** 提供全屏覆盖的对话界面，状态变化实时同步到 Gateway。
5. **GatewayConnection** 使用 Swift actor 模型封装 WebSocket 通信，支持 20+ 种 RPC 方法。
6. **权限管理**覆盖 8 种系统权限，通过 `PermissionManager` 统一处理请求和状态检查。